

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Departamento de Ciencias de la Computación

CC3069 — Programación Paralela y Distribuida · Sección 20

Ing. Santiago Solórzano



Proyecto 1 - Screensaver

Paralelización de una simulación gráfica con OpenMP

Ricardo Arturo Godínez Sánchez - 23247

Vianka Vanessa Castro Ordoñez - 23201

Abby Sofia Donis Agreda - 22440

GUATEMALA, 31 de agosto de 2026

Contenido

Contenido	2
Resumen	4
Introducción	4
Antecedentes y fundamentos	4
Objetivos	5
Objetivo general	5
Objetivos específicos	5
Descripción y alcance del proyecto	5
Diseño de la simulación	5
Alcance implementado	6
Requisitos del hito inicial	6
Arquitectura del repositorio	7
Flujo de ejecución	7
Implementación secuencial y paralela	8
Una sola base de código	8
Pasada A: interacción entre vacas	8
Pasada B: integración y rebote	8
Sincronización y memoria compartida	8
Estrellas: experimento opcional	9
Programación defensiva	9
Presentación gráfica actual	9
Entorno de ejecución y compilación	9
Metodología	11
Variables y métricas	11
Protocolo por integrante	11
Experimentos automatizados	12
Criterios de validez	12
Resultados	12
Resultados de Vianka Vanessa Castro Ordoñez	13

Reproducibilidad y archivos	13
Resumen de mediciones	13
Resultados de Ricardo Arturo Godínez Sánchez	15
Reproducibilidad y archivos	15
Resumen de mediciones	15
Resultados de Abby Sofia Donis Agreda.....	17
Reproducibilidad y archivos	17
Resumen de mediciones	17
Resultados consolidados	19
Speedup y eficiencia de vacas	19
Scheduling	19
False sharing.....	19
Paralelización opcional de estrellas	20
Análisis y discusión	21
Conclusiones finales del equipo	23
Recomendaciones	23
Apéndices	24
Apéndice D. Lista de verificación para la entrega	24
Anexos	25
Anexo 1 — Diagrama de flujo final.....	25
Anexo 2 — Catálogo de funciones	25
Anexo 3 — Bitácora de pruebas.....	26
Anexo 4 — Repositorio.....	26
Bibliografía.....	27

Nota de edición: después de agregar resultados o mover secciones, seleccione todo el documento (Ctrl+A) y presione F9 para actualizar títulos y números de página.

Resumen

ZipZip es un screensaver 3D en C++ que representa una granja suspendida en el espacio. La escena incluye vacas animadas, un campo de estrellas, estrellas fugaces, planetas con texturas procedurales y aros, una plataforma con textura de grama y un OVNI-gato cuyo cuerpo metálico y piloto verde neón se renderizan por grupos del archivo OBJ. SDL2 administra la ventana y los eventos; OpenGL realiza todo el dibujo en el hilo principal.

La evaluación académica se concentra en la simulación de las vacas. El mismo ejecutable ofrece modo serial y paralelo. OpenMP se aplica a dos pasadas: una interacción $O(N^2)$ entre vacas y una integración $O(N)$. Cada iteración escribe posiciones auxiliares o estados disjuntos; una reducción min calcula la distancia mínima al OVNI y la barrera implícita entre pasadas conserva el orden de dependencias. El modo `--bench` excluye SDL/OpenGL, usa un dt fijo y registra tiempos comparables.

El repositorio ya automatiza diez repeticiones por configuración y calcula promedio, desviación estándar, speedup y eficiencia. Este informe deja sin completar deliberadamente las tablas de mediciones, porque deben llenarse con ejecuciones reales de los tres integrantes sobre el commit acordado.

Introducción

La programación paralela busca aprovechar varios núcleos para reducir el tiempo de una carga divisible. En memoria compartida, OpenMP permite expresar el reparto del trabajo mediante directivas sobre ciclos, pero el rendimiento depende de la fracción serial, el balance, la sincronización, la localidad y el costo de crear o coordinar hilos [1, 2, 4, 9].

Una simulación gráfica permite distinguir dos responsabilidades. La actualización del estado puede procesar entidades independientes en paralelo; en cambio, SDL2 y OpenGL mantienen la creación del contexto, los eventos y el renderizado en el hilo principal. Esta separación evita llamadas gráficas concurrentes y permite medir el kernel de simulación sin confundirlo con el costo de dibujo.

Antecedentes y fundamentos

- Speedup: $S(T) = t_{\text{serial}} / t_{\text{paralelo}}$. Un valor mayor que 1 indica mejora respecto de la referencia serial.
- Eficiencia: $E(T) = S(T) / T$. Expresa qué fracción del speedup ideal aporta cada hilo.
- Ley de Amdahl: la fracción serial limita el speedup cuando el tamaño del problema se mantiene fijo [1].
- Ley de Gustafson: al aumentar el problema, el trabajo paralelo puede dominar y justificar más procesadores [4].

- False sharing: hilos que escriben datos diferentes pero cercanos pueden invalidar la misma línea de caché; el experimento con chunk 1 busca hacerlo visible [9].
- Scheduling: static, dynamic y guided distribuyen iteraciones con costos y equilibrio distintos. La elección debe justificarse con mediciones.

Objetivos

Objetivo general

Implementar y evaluar una simulación gráfica serial y paralela con OpenMP, manteniendo el renderizado en el hilo principal y cuantificando el efecto del número de vacas, hilos, scheduling y distribución de memoria sobre el rendimiento.

Objetivos específicos

- Conservar una única base de código capaz de ejecutar el mismo algoritmo en modo serial o paralelo.
- Paralelizar primero la actualización de las vacas y comprobar que no introduce condiciones de carrera.
- Separar el tiempo de simulación del renderizado y ofrecer un benchmark reproducible sin ventana.
- Comparar tiempos, speedup y eficiencia con al menos diez repeticiones por configuración.
- Evaluar el efecto de scheduling y chunk, incluido un caso destinado a observar false sharing.
- Medir por separado si la paralelización de estrellas compensa su overhead.
- Documentar hardware, software, comandos, resultados y conclusiones verificables de cada integrante.

Descripción y alcance del proyecto

Diseño de la simulación

La escena final mantiene la idea de una granja espacial, pero su implementación actual difiere del concepto inicial: la plataforma es una cúpula semicircular vista desde arriba; las vacas rebotan, se separan y conservan cohesión; el OVNI recorre una trayectoria de Lissajous y no captura vacas todavía. No existe un sistema de partículas de captura, por lo que el informe no lo presenta como terminado.



Figura 1. Concepto visual original del proyecto; no representa exactamente todas las funciones actuales.

Alcance implementado

Subsistema	Estado actual	Tratamiento paralelo
Ventana y eventos	SDL2/OpenGL, 900×700 por defecto; 640×480 mínimo; cierre con Esc o X.	Hilo principal.
Vacas	N configurable; posición, velocidad, giro, escala, color, rebote, separación y cohesión.	Dos pasadas seriales o OpenMP.
Estrellas	M configurable; parpadeo, movimiento y estrellas fugaces.	Serial por defecto; experimento opcional.
OVNI-gato	Trayectoria de Lissajous; nave metálica y gato verde neón por grupos OBJ.	Actualización $O(1)$, serial.
Planetas	Cuatro planetas, texturas procedurales, rotación y aros.	Serial; fuera del kernel evaluado.
Fondo y grama	BMP cargados con SDL2; fallback a colores sólidos si fallan.	Carga y render en hilo principal.
Medición	HUD y modo --bench; CSV y resumen estadístico.	Cronometraje con steady_clock.
Captura/partículas	No implementado en el commit de referencia.	Trabajo futuro; no se evalúa.

Requisitos del hito inicial

Requisito	Evidencia actual
Ventana gráfica y API elegida	SDL2 crea ventana/contexto; OpenGL renderiza la escena.

Requisito	Evidencia actual
Variable en memoria	Escena::vacas, Escena::ax/ay y CampoEstrellas::estrellas almacenan entidades y auxiliares.
Render de un elemento	Renderer::dibujarVacas carga cow.obj y dibuja cada instancia.
Función de actualización	actualizarEscena determina interacción, integración, rebote y giro.
Commit identificable	El historial conserva hitos; para este informe se fija abb0305 como referencia.

Arquitectura del repositorio

```

Proyecto1_Paralela_Zip/
├── CMakeLists.txt
├── assets/
│   ├── models/      cow.obj, ufo_gato.obj, planet.obj
│   └── textures/     space_background.bmp, grass.bmp
├── include/zipzip/   interfaces de core, assets, simulation y rendering
├── src/
│   ├── main.cpp      CLI, eventos, ciclo gráfico y --bench
│   ├── assets/       cargador OBJ
│   ├── simulation/   escena, vacas, OVNI, planetas y estrellas
│   └── rendering/    OpenGL, texturas y HUD
├── scripts/          benchmark.sh y resumir_benchmark.py
└── docs/             propuesta, diagramas, matemática y anexos

```

Capa	Responsabilidad	Dependencias relevantes
Aplicación	Validar argumentos, elegir modo, eventos, loop principal y benchmark.	SDL2 solo en modo gráfico; OpenMP runtime.
Simulación	Crear y actualizar estado determinista.	C++ estándar y OpenMP; no depende de SDL/OpenGL.
Renderizado	Dibujar fondo, estrellas, plataforma, vacas, OVNI, planetas y HUD.	OpenGL y SDL2; siempre hilo principal.
Recursos	Cargar y normalizar OBJ; conservar rangos g/o para materiales visuales.	C++ estándar.
Núcleo	RNG, cámara y cronómetro compartidos.	Headers reutilizables.
Automatización	Repetir experimentos y resumir CSV.	Bash y Python 3.

Flujo de ejecución

1. Configuración	2. Estado	3. Simulación	4. Salida
Leer/validar CLI Configurar OpenMP	Reservar vacas, ax/ay y estrellas Crear escena con semilla	Actualizar OVNI/planetas Pasada A → barrera → Pasada B Actualizar estrellas	Gráfico: render + HUD Benchmark: fila CSV

El diagrama detallado se mantiene en docs/anexo_01_diagrama_flujo.md. Ambas modalidades recorren el mismo flujo; la condición usarOpenMP decide si los pragmas ejecutan más de un hilo.

Implementación secuencial y paralela

Una sola base de código

No existen dos proyectos duplicados. La opción `--modo serial|paralelo` controla el argumento `if(usarOpenMP)` de las directivas. Una compilación con `ZIPZIP_OPENMP=OFF` sirve como referencia adicional y rechaza de forma explícita la solicitud de modo paralelo.

Pasada A: interacción entre vacas

La pasada A evalúa la separación para cada par de vacas, por lo que su complejidad es $O(N^2)$. La iteración i solo lee el vector completo de vacas y escribe `ax[i]` y `ay[i]`. La distancia mínima al OVNI se combina con una reducción; no altera el movimiento.

```
#pragma omp parallel for if(usarOpenMP) schedule(runtime) \
    reduction(min:distanciaMinimaOvni)
for (long i = 0; i < cantidadVacas; ++i) {
    // lee vacas[j]; escribe exclusivamente ax[i], ay[i]
}
```

Pasada B: integración y rebote

La barrera implícita al terminar la pasada A garantiza que todas las aceleraciones estén disponibles. En la pasada B, cada iteración modifica una Instancia diferente, aplica cohesión, limita rapidez, actualiza giro y resuelve el rebote dentro de la cúpula. Es $O(N)$ y tiene menor intensidad computacional.

```
#pragma omp parallel for if(usarOpenMP) schedule(runtime)
for (long i = 0; i < cantidadVacas; ++i) {
    // lee ax[i], ay[i]; actualiza exclusivamente vacas[i]
}
```

Sincronización y memoria compartida

Dato	Acceso concurrente	Protección/razón
Escena::vacas en pasada A	Solo lectura.	No requiere protección.
Escena::ax[i], ay[i]	Un índice por iteración.	Escrituras disjuntas.
distanciaMinimaOvni	Mínimo global.	reduction(min:...).
Dependencia $A \rightarrow B$	B consume aceleraciones calculadas en A.	Barrera implícita al final del primer parallel for.

Dato	Acceso concurrente	Protección/razón
Escena::vacas[i] en pasada B	Una vaca por iteración.	Escrituras disjuntas; no requiere critical ni mutex.
SDL/OpenGL/Renderer	Contexto gráfico compartido.	Se limita al hilo principal.

Estrellas: experimento opcional

Las estrellas se mantienen seriales por defecto. Con $M=180$, cada iteración hace poco trabajo y el overhead de OpenMP puede superar el ahorro. La bandera `--estrellas-paralelas` existe únicamente para medir esa hipótesis con $M=180$ y un M grande, por ejemplo 10 000; la decisión final debe basarse en las tablas del equipo.

Programación defensiva

- Rangos explícitos: vacas 1..100000, estrellas 1..1000000, hilos 1..1024, ancho 640..7680 y alto 480..4320.
- Rechazo de enteros parciales, semilla cero, opciones desconocidas, `schedule` inválido y argumentos incompletos.
- Mensaje claro si se pide OpenMP en una compilación serial o estrellas paralelas en modo serial.
- Carga de texturas con fallback; un BMP ausente no derriba toda la aplicación.
- Semilla reproducible y `--dump-estado` para contrastar resultados seriales/paralelos.

Presentación gráfica actual

Elemento	Implementación
Fondo	space_background.bmp como quad ortográfico; estrellas encima.
Plataforma	Cúpula semicircular con grass.bmp repetible.
Vacas	cow.obj con transformaciones y colores por instancia.
OVNI-gato	ufo_gato.obj conserva grupos g/o; nave metálica y gato verde neón.
Planetas	planet.obj, cuatro texturas procedurales, rotación y aros.
HUD	FPS, tiempo SIM, hilos y distancia mínima al OVNI.

Entorno de ejecución y compilación

Requisitos

- CMake 3.20 o posterior y compilador con C++17.
- SDL2 y OpenGL.

- OpenMP para el modo paralelo; puede deshabilitarse con ZIPZIP_OPENMP=OFF.
- Bash para benchmark.sh y Python 3 para resumir_benchmark.py.

Windows — MSYS2 UCRT64

```
pacman -S --needed mingw-w64-ucrt-x86_64-gcc \
  mingw-w64-ucrt-x86_64-cmake mingw-w64-ucrt-x86_64-SDL2 \
  mingw-w64-ucrt-x86_64-ninja

cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build
./build/zipzip.exe
```

Importante en UCRT64

La barra invertida solo continúa una orden cuando queda al final de la línea. No escriba “\ main.cpp”. Con CMake no es necesario memorizar la lista de archivos ni bibliotecas.

Linux

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
./build/zipzip
```

Compilación serial adicional

```
cmake -S . -B build-serial -DZIPZIP_OPENMP=OFF -DCMAKE_BUILD_TYPE=Release
cmake --build build-serial -j
```

Opciones relevantes

Opción	Default	Uso
N o --vacas N	20	Cantidad de vacas.
--estrellas N	180	Cantidad de estrellas.
--modo serial paralelo	paralelo*	Activa/desactiva OpenMP en la simulación.
--hilos N	runtime	Número de hilos.
--schedule T	static	static, dynamic o guided.
--chunk N	0	Bloque automático o explícito.
--semilla N	1234	Escena reproducible.
--bench --frames N	apagado / 300	Medición sin ventana.
--dump-estado	apagado	Estado final para comparación exacta.
--estrellas-paralelas	apagado	Experimento dedicado de estrellas.
--ancho / --alto	900 / 700	Dimensiones del canvas.

Opción	Default	Uso
--modelo ruta	assets/models/cow.obj	Modelo OBJ de vacas.

** En una compilación sin OpenMP, el modo predeterminado es serial.*

Verificación determinista

```
./build/zipzip --bench --modo serial --vacas 3000 --frames 30 \
--semilla 1234 --dump-estado > serial.txt
./build/zipzip --bench --modo paralelo --vacas 3000 --frames 30 \
--semilla 1234 --hilos 8 --dump-estado > paralelo.txt
diff serial.txt paralelo.txt
```

Resultado esperado: diff no imprime diferencias. En Windows UCRT64, los mismos comandos funcionan con ./build/zipzip.exe.

Metodología

Regla académica

Cada configuración debe tener al menos 10 mediciones. Una sola corrida sirve para diagnosticar, no para sostener conclusiones.

Variables y métricas

Tipo	Variables
Independientes	N vacas; M estrellas; T hilos; modo; schedule; chunk; equipo.
Controladas	Commit, Release, semilla 1234, frames, aplicaciones en segundo plano y perfil de energía.
Dependientes	ms_sim_avg, ms_sim_p95, ms_pasada_a_avg, ms_pasada_b_avg y FPS derivado.
Calculadas	Promedio, desviación estándar, speedup y eficiencia.

Protocolo por integrante

1. Abrir una terminal en la raíz del repositorio y confirmar el commit con git rev-parse --short HEAD.
2. Compilar en Release con OpenMP y ejecutar ./build/zipzip --ayuda.
3. Ejecutar una prueba visual y guardar una captura con HUD visible.
4. Comprobar equivalencia serial/paralela con --dump-estado y la misma semilla.
5. Ejecutar benchmark.sh sin reducir REPETICIONES por debajo de 10.
6. Generar resumen.csv con resumir_benchmark.py.

7. Guardar CSV, capturas y observaciones en una carpeta identificada con el nombre del integrante.
8. Copiar al informe los agregados; no transcribir a mano todas las filas crudas.

```
bash scripts/benchmark.sh build/zipzip > resultados.csv
python scripts/resumir_benchmark.py resultados.csv > resumen.csv
```

En Windows UCRT64, use build/zipzip.exe como argumento del script.

Experimentos automatizados

Experimento	Configuraciones predeterminadas	Pregunta
Speedup	N={1000,3000,5000}; T={1,2,4,8}; static.	¿Cómo escala la actualización de vacas?
Scheduling	N=5000; static, dynamic, guided; T máximo.	¿Compensa el balance dinámico su overhead?
False sharing	N=3000; static con chunk=0 y chunk=1.	¿Las escrituras intercaladas degradan la pasada B?
Estrellas	M={180,10000}; serial, paralelo sin y con estrellas OpenMP.	¿Cuándo conviene paralelizar estrellas?

Criterios de validez

- Usar una build Release y el mismo commit para toda comparación dentro de un equipo.
- Ejecutar 30 frames de calentamiento internos antes de los frames medidos, como hace --bench.
- No mezclar equipos en una razón de speedup: la base serial debe provenir del mismo equipo y experimento.
- Reportar anomalías, thermal throttling, procesos en segundo plano y cambios de perfil de energía.
- Conservar resultados que contradigan la hipótesis; analizarlos en vez de descartarlos sin justificación.

Resultados

Esta sección conserva el espacio de resultados del informe original. Cada integrante debe completar su equipo, comandos, archivos, al menos diez mediciones por configuración, capturas y observaciones; después se consolidan los valores para el análisis.

Resultados de Vianka Vanessa Castro Ordoñez

Carné: 23201. Complete esta sección en el equipo donde se ejecutaron las pruebas; no copie los datos de otro integrante.

Dato	Valor observado
Equipo	HP EliteBook [VERIFICAR modelo exacto]
CPU	Intel Core i7 de 11.ª generación [VERIFICAR modelo]
Núcleos / hilos lógicos	4 / 8 [VERIFICAR]
Sistema operativo	Windows 11 Pro [AGREGAR versión]
RAM	[POR COMPLETAR: capacidad y velocidad si se conoce]
GPU y controlador	[POR COMPLETAR]
Compilador	[POR COMPLETAR: g++ --version]
CMake / Ninja	[POR COMPLETAR: versiones]
OpenMP	[POR COMPLETAR: _OPENMP/runtime]
Commit	[POR COMPLETAR; esperado abb0305 o commit final acordado]
Perfil de energía	[POR COMPLETAR]
Fecha y hora	[POR COMPLETAR]

Reproducibilidad y archivos

Evidencia	Valor / ruta
Comando de compilación	[POR COMPLETAR]
Comando de benchmark	[POR COMPLETAR]
CSV crudo	[POR COMPLETAR: ruta o enlace]
CSV resumido	[POR COMPLETAR: ruta o enlace]
Cantidad de mediciones/configuración	[POR COMPLETAR; mínimo 10]
Resultado de diff	[POR COMPLETAR: sin diferencias / explicar]

Resumen de mediciones

Exp.	N/M	Configuración	Prom.	Desv.	Speedup	Efic.
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[x]	[%]
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[x]	[%]
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[x]	[%]
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[x]	[%]

Exp.	N/M	Configuración	Prom.	Desv.	Speedup	Efic.
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[×]	[%]
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[×]	[%]
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[×]	[%]
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[×]	[%]

Captura de ejecución visual

Inserte una captura legible de la ventana con HUD y anote el comando utilizado.

Observaciones del integrante

Describa estabilidad, ruido, temperatura, anomalías, mejor configuración y cualquier limitación.

Resultados de Ricardo Arturo Godínez Sánchez

Carné: 23247. Complete esta sección en el equipo donde se ejecutaron las pruebas; no copie los datos de otro integrante.

Dato	Valor observado
Equipo	Lenovo Yoga 6 [VERIFICAR modelo exacto]
CPU	AMD Ryzen 7 5700U con Radeon Graphics
Núcleos / hilos lógicos	8 / 16 [VERIFICAR]
Sistema operativo	Arch Linux [AGREGAR kernel/versión]
RAM	[POR COMPLETAR: capacidad y velocidad si se conoce]
GPU y controlador	[POR COMPLETAR]
Compilador	[POR COMPLETAR: g++ --version]
CMake / Ninja	[POR COMPLETAR: versiones]
OpenMP	[POR COMPLETAR: _OPENMP/runtime]
Commit	[POR COMPLETAR; esperado abb0305 o commit final acordado]
Perfil de energía	[POR COMPLETAR]
Fecha y hora	[POR COMPLETAR]

Reproducibilidad y archivos

Evidencia	Valor / ruta
Comando de compilación	[POR COMPLETAR]
Comando de benchmark	[POR COMPLETAR]
CSV crudo	[POR COMPLETAR: ruta o enlace]
CSV resumido	[POR COMPLETAR: ruta o enlace]
Cantidad de mediciones/configuración	[POR COMPLETAR; mínimo 10]
Resultado de diff	[POR COMPLETAR: sin diferencias / explicar]

Resumen de mediciones

Exp.	N/M	Configuración	Prom.	Desv.	Speedup	Efic.
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[×]	[%]
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[×]	[%]
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[×]	[%]
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[×]	[%]

Exp.	N/M	Configuración	Prom.	Desv.	Speedup	Efic.
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[×]	[%]
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[×]	[%]
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[×]	[%]
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[×]	[%]

Captura de ejecución visual

Inserte una captura legible de la ventana con HUD y anote el comando utilizado.

Observaciones del integrante

Describe estabilidad, ruido, temperatura, anomalías, mejor configuración y cualquier limitación.

Resultados de Abby Sofia Donis Agreda

Carné: 22440. Complete esta sección en el equipo donde se ejecutaron las pruebas; no copie los datos de otro integrante.

Dato	Valor observado
Equipo	Lenovo ThinkPad [VERIFICAR modelo exacto]
CPU	Intel Core i7 de 8.ª generación [VERIFICAR modelo]
Núcleos / hilos lógicos	4 / 8 [VERIFICAR]
Sistema operativo	Windows 11 Pro [AGREGAR versión]
RAM	[POR COMPLETAR: capacidad y velocidad si se conoce]
GPU y controlador	[POR COMPLETAR]
Compilador	[POR COMPLETAR: g++ --version]
CMake / Ninja	[POR COMPLETAR: versiones]
OpenMP	[POR COMPLETAR: _OPENMP/runtime]
Commit	[POR COMPLETAR; esperado abb0305 o commit final acordado]
Perfil de energía	[POR COMPLETAR]
Fecha y hora	[POR COMPLETAR]

Reproducibilidad y archivos

Evidencia	Valor / ruta
Comando de compilación	[POR COMPLETAR]
Comando de benchmark	[POR COMPLETAR]
CSV crudo	[POR COMPLETAR: ruta o enlace]
CSV resumido	[POR COMPLETAR: ruta o enlace]
Cantidad de mediciones/configuración	[POR COMPLETAR; mínimo 10]
Resultado de diff	[POR COMPLETAR: sin diferencias / explicar]

Resumen de mediciones

Exp.	N/M	Configuración	Prom.	Desv.	Speedup	Efic.
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[×]	[%]
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[×]	[%]
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[×]	[%]
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[×]	[%]

Exp.	N/M	Configuración	Prom.	Desv.	Speedup	Efic.
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[×]	[%]
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[×]	[%]
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[×]	[%]
[Exp.]	[N/M]	[Modo/T/schedule/chunk]	[ms]	[ms]	[×]	[%]

Captura de ejecución visual

Inserte una captura legible de la ventana con HUD y anote el comando utilizado.

Observaciones del integrante

Describe estabilidad, ruido, temperatura, anomalías, mejor configuración y cualquier limitación.

Resultados consolidados

Datos aún no incorporados

Los valores obtenidos durante validaciones rápidas de desarrollo no se presentan como resultados académicos. Esta sección debe llenarse con las mediciones completas de los tres integrantes, cada una con al menos diez repeticiones y evidencia guardada.

Speedup y eficiencia de vacas

Equipo	N	T	t serial	t paralelo	Speedup	Eficiencia
Vianka	1000	[T]	[serial ms]	[paralelo ms]	[x]	[%]
Vianka	3000	[T]	[serial ms]	[paralelo ms]	[x]	[%]
Vianka	5000	[T]	[serial ms]	[paralelo ms]	[x]	[%]
Ricardo	1000	[T]	[serial ms]	[paralelo ms]	[x]	[%]
Ricardo	3000	[T]	[serial ms]	[paralelo ms]	[x]	[%]
Ricardo	5000	[T]	[serial ms]	[paralelo ms]	[x]	[%]
Abby	1000	[T]	[serial ms]	[paralelo ms]	[x]	[%]
Abby	3000	[T]	[serial ms]	[paralelo ms]	[x]	[%]
Abby	5000	[T]	[serial ms]	[paralelo ms]	[x]	[%]

Scheduling

Equipo	N	T	static	dynamic	guided	Mejor
Vianka	5000	[T máx.]	[ms]	[ms]	[ms]	[tipo]
Ricardo	5000	[T máx.]	[ms]	[ms]	[ms]	[tipo]
Abby	5000	[T máx.]	[ms]	[ms]	[ms]	[tipo]

False sharing

Equipo	N	T	chunk=0	chunk=1	Cambio %	Interpretación
Vianka	3000	[T máx.]	[ms]	[ms]	[%]	[POR COMPLETAR]
Ricardo	3000	[T máx.]	[ms]	[ms]	[%]	[POR COMPLETAR]
Abby	3000	[T máx.]	[ms]	[ms]	[%]	[POR COMPLETAR]

Paralelización opcional de estrellas

Equipo	M	Base	Estrellas OpenMP	Conclusión
Vianka	180	[paralelo sin estrellas OMP]	[paralelo con estrellas OMP]	[conviene/no]
Vianka	10000	[paralelo sin estrellas OMP]	[paralelo con estrellas OMP]	[conviene/no]
Ricardo	180	[paralelo sin estrellas OMP]	[paralelo con estrellas OMP]	[conviene/no]
Ricardo	10000	[paralelo sin estrellas OMP]	[paralelo con estrellas OMP]	[conviene/no]
Abby	180	[paralelo sin estrellas OMP]	[paralelo con estrellas OMP]	[conviene/no]
Abby	10000	[paralelo sin estrellas OMP]	[paralelo con estrellas OMP]	[conviene/no]

Gráfica 1 — Speedup vs. número de hilos

Inserte una gráfica por N o por equipo. Debe incluir título, ejes con unidades, leyenda y referencia en el texto.

Gráfica 2 — Eficiencia vs. número de hilos

Inserte una gráfica comparable con la anterior y explique la pérdida de eficiencia.

Gráfica 3 — Scheduling / chunk / estrellas

Use una o más gráficas solo si aportan una comparación clara; indique promedio y variabilidad.

Análisis y discusión

Esta sección debe relacionar los valores consolidados con la arquitectura y no limitarse a repetir las tablas. Complete cada apartado con números concretos y mencione el equipo, N y T de la observación.

Escalabilidad

¿Para qué N aparece speedup? ¿En qué T se aplana o retrocede? Compare con Amdahl y con la intensidad $O(N^2)$ de la pasada A.

Eficiencia

¿Cómo cambia $E(T)$? ¿La caída se explica por trabajo serial, overhead, memoria, frecuencia/temperatura o variabilidad?

Pasadas A y B

Compare $ms_pasada_a_avg$ y $ms_pasada_b_avg$. ¿Cuál domina y cuál escala mejor?

Scheduling

¿Static fue suficiente por el costo uniforme? Si dynamic/guided cambia el tiempo, cuantifique el cambio.

False sharing

¿ $chunk=1$ empeoró la pasada B o el total? Explique la relación con Instancia de 40 bytes y líneas de caché de 64 bytes como hipótesis, no como hecho no medido.

Estrellas

¿El overhead domina con $M=180$? ¿Existe un umbral donde --estrellas-paralelas mejora?

Diferencias entre equipos

Compare arquitectura de CPU, núcleos/hilos, sistema, compilador y variabilidad; no compare tiempos absolutos sin contexto.

Validez

Identifique amenazas: procesos en segundo plano, turbo, throttling, perfiles de energía, versiones y precisión del cronómetro.

Conclusiones finales del equipo

1. [POR COMPLETAR: conclusión principal con el mejor speedup, N, T y equipo.]
2. [POR COMPLETAR: conclusión sobre eficiencia y límite de escalamiento.]
3. [POR COMPLETAR: conclusión sobre scheduling y false sharing.]
4. [POR COMPLETAR: conclusión sobre paralelizar o no las estrellas.]
5. [POR COMPLETAR: conclusión sobre corrección, sincronización y renderizado en hilo principal.]

Recomendaciones

- Mantener --modo serial y --modo paralelo dentro del mismo ejecutable para evitar divergencias funcionales.
- Conservar Release, semilla, frames y commit en toda comparación.
- No paralelizar OpenGL/SDL2 con este diseño; priorizar kernels de simulación medibles.
- Evaluar estructuras espaciales solo como trabajo futuro: reducirían $O(N^2)$, pero cambiarían el experimento académico actual.
- [POR COMPLETAR: recomendación basada en el mejor T observado por el equipo.]

Apéndices

Apéndice D. Lista de verificación para la entrega

Estado actual de cumplimiento

Requisito	Estado	Evidencia / acción
Versión secuencial funcional	IMPLEMENTADO	--modo serial en el mismo ejecutable.
Versión OpenMP funcional	IMPLEMENTADO	Dos parallel for en actualización de vacas.
Comparación sin duplicar proyecto	IMPLEMENTADO	Condición if(usarOpenMP).
Número de threads	IMPLEMENTADO	--hilos y HUD/CSV.
Programación defensiva	IMPLEMENTADO	Rangos, errores, fallback y builds sin OpenMP.
Sincronización	IMPLEMENTADO	Escrituras disjuntas, reducción y barrera implícita.
Render en hilo principal	IMPLEMENTADO	SDL/OpenGL/Renderer no se llaman desde OpenMP.
Mediciones ≥ 10	AUTOMATIZADO	benchmark.sh obliga REPETICIONES ≥ 10 ; falta ejecutar por integrante.
Speedup y eficiencia	AUTOMATIZADO	resumir_benchmark.py; falta consolidar resultados.
Schedules y chunk	AUTOMATIZADO	Experimentos dedicados; falta análisis.
Estrellas	IMPLEMENTADO/EXPERIMENTO	Serial por defecto, --estrellas-paralelas opcional.
Anexo 1	IMPLEMENTADO	docs/anexo_01_diagrama_flujo.md; resumen incorporado.
Anexo 2	IMPLEMENTADO	docs/anexo_02_catalogo_funciones.md; catálogo incorporado.
Anexo 3	POR COMPLETAR	Adjuntar bitácoras, CSV y capturas reales.
Conclusiones con datos	POR COMPLETAR	Rellenar después de consolidar mediciones.

Lista de verificación antes de entregar

- El código compila desde cero siguiendo el README.
- El commit evaluado es el mismo en los tres equipos o se explican las diferencias.
- Serial y paralelo producen el mismo --dump-estado con semilla idéntica.
- Cada configuración tiene al menos diez mediciones y unidades visibles.
- Las gráficas tienen título, ejes, leyenda y referencia en el texto.
- Las conclusiones citan valores concretos y no describen solamente la metodología.
- Se documentan CPU, núcleos, hilos, RAM, OS, compilador, flags, T, scheduling, N, M y seed.
- No quedan marcas POR COMPLETAR salvo que el docente permita explícitamente un borrador.
- La tabla de contenido y los números de página fueron actualizados con Ctrl+A, F9.

Anexos

Anexo 1 — Diagrama de flujo final

```
Inicio → validar argumentos → configurar OpenMP → crear estado
├ --bench: calentamiento → medir frames → CSV
└ gráfico: SDL/OpenGL → eventos → simular → renderizar → HUD

Simular: OVNI/planetas serial → pasada A serial/OMP → barrera
        → pasada B serial/OMP → estrellas serial/OMP opcional
```

Fuente editable completa: `docs/anexo_01_diagrama_flujo.md`. Antes de entregar, puede exportarse el bloque Mermaid como imagen e insertarse debajo.

Exportación del diagrama

Inserte aquí la versión gráfica final del diagrama Mermaid si la rúbrica exige una figura.

Anexo 2 — Catálogo de funciones

Función	Entradas	Salida	Responsabilidad
leerArgumentos	argc/argv	bool	Valida CLI y rangos.
configurarOpenMP	Opciones	hilos	Configura threads y schedule.
ejecutarBench	Opciones	código	Calienta, mide y emite CSV.
main	argumentos	código	Coordina benchmark o modo gráfico.
Cronometro::*	muestras	estadísticas	Promedio, mínimo, máximo y p95.
xorshift32 / Rng	semilla/estado	aleatorio	Escena determinista.
cargarOBJ	ruta/modelo	bool	Triangula, normaliza y conserva grupos.
crearEscena	N, tamaño, seed	—	Reserva e inicializa entidades.
actualizarEscena	Escena, dt, modo	—	Pasadas A y B; serial/OpenMP.
crearCampoEstrellas	M, tamaño, seed	—	Inicializa estrellas y fugaces.
actualizarCampoEstrellas	Campo, dt, modo	—	Actualiza brillo/movimiento.

Función	Entradas	Salida	Responsabilidad
Renderer::inicializar	contexto	—	Configura OpenGL y recursos.
cargarTexturaBMP	ruta/modo	bool	Sube BMP o permite fallback.
Renderer::dibujar	modelos/estado/métricas	—	Compone el frame.
Renderer::dibujarVacas	modelo/escena	—	Dibuja instancias.
Renderer::dibujarOvni	modelo/escena	—	Aplica dos acabados por grupos.
Renderer::dibujarPlanetas	escena	—	Texturas, rotación y aros.
Renderer::dibujarHUD	métricas	—	Muestra FPS/SIM/hilos/mínimo.
benchmark.sh	binario/variables	CSV	≥10 repeticiones/configuración.
resumir_benchmark.py	CSV crudo	CSV resumen	Promedio, stdev, speedup, eficiencia.

El catálogo ampliado y mantenible está en `docs/anexo_02_catálogo_funciones.md`.

Anexo 3 — Bitácora de pruebas

ID	Integrante	Fecha	Commit	Experimento	Archivo/evidencia	Estado
1	[Nombre]	[AAAA-MM-DD]	[hash]	[tipo]	[ruta/enlace]	[OK/nota]
2	[Nombre]	[AAAA-MM-DD]	[hash]	[tipo]	[ruta/enlace]	[OK/nota]
3	[Nombre]	[AAAA-MM-DD]	[hash]	[tipo]	[ruta/enlace]	[OK/nota]
4	[Nombre]	[AAAA-MM-DD]	[hash]	[tipo]	[ruta/enlace]	[OK/nota]
5	[Nombre]	[AAAA-MM-DD]	[hash]	[tipo]	[ruta/enlace]	[OK/nota]
6	[Nombre]	[AAAA-MM-DD]	[hash]	[tipo]	[ruta/enlace]	[OK/nota]
7	[Nombre]	[AAAA-MM-DD]	[hash]	[tipo]	[ruta/enlace]	[OK/nota]
8	[Nombre]	[AAAA-MM-DD]	[hash]	[tipo]	[ruta/enlace]	[OK/nota]
9	[Nombre]	[AAAA-MM-DD]	[hash]	[tipo]	[ruta/enlace]	[OK/nota]
10	[Nombre]	[AAAA-MM-DD]	[hash]	[tipo]	[ruta/enlace]	[OK/nota]
11	[Nombre]	[AAAA-MM-DD]	[hash]	[tipo]	[ruta/enlace]	[OK/nota]
12	[Nombre]	[AAAA-MM-DD]	[hash]	[tipo]	[ruta/enlace]	[OK/nota]

Anexo 4 — Repositorio

Repositorio GitHub de ZipZip

Commit documentado al preparar esta versión: abb0305 (update paralelism). Si el equipo entrega otro commit o tag, reemplázelo en el índice, la metodología y las bitácoras.

Bibliografía

- [1] Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. AFIPS Spring Joint Computer Conference, 30, 483–485. <https://doi.org/10.1145/1465482.1465560>
- [2] Dagum, L., & Menon, R. (1998). OpenMP: An industry standard API for shared-memory programming. IEEE Computational Science & Engineering, 5(1), 46–55. <https://doi.org/10.1109/99.660313>
- [3] Flynn, M. J. (1972). Some computer organizations and their effectiveness. IEEE Transactions on Computers, C-21(9), 948–960. <https://doi.org/10.1109/TC.1972.5009071>
- [4] Gustafson, J. L. (1988). Reevaluating Amdahl's Law. Communications of the ACM, 31(5), 532–533. <https://doi.org/10.1145/42411.42415>
- [5] Khronos Group. (2022). OpenGL 4.6 Core Profile Specification. <https://registry.khronos.org/OpenGL/specs/gl/glspec46.core.pdf>
- [6] OpenMP Architecture Review Board. (2021). OpenMP Application Programming Interface, Version 5.2. <https://www.openmp.org/spec-html/5.2/openmp.html>
- [7] Reynolds, C. W. (1987). Flocks, herds and schools: A distributed behavioral model. ACM SIGGRAPH Computer Graphics, 21(4), 25–34. <https://doi.org/10.1145/37402.37406>
- [8] SDL contributors. (s. f.). FAQ: Development. SDL2 Wiki. Recuperado el 20 de agosto de 2026 de <https://wiki.libsdl.org/SDL2/FAQDevelopment>
- [9] Williams, S., Waterman, A., & Patterson, D. (2009). Roofline: An insightful visual performance model for multicore architectures. Communications of the ACM, 52(4), 65–76. <https://doi.org/10.1145/1498765.1498785>